



CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-G3

System Integration and Architecture

System Design Document (SDD)

Project Title: Vigilo - Visitor Log System

Prepared By: Basalo, Kurt Derrick G.

Version: 1.0

Date: February 28, 2026

Status: Final

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
0.1	Feb. 21, 2026	Kurt Derrick G. Basalo	Initial draft	Draft
1.0	Feb. 28, 2026	Kurt Derrick G. Basalo	Completed other sections of the document	Review

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY	4
1.0 INTRODUCTION	5
2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION	5
3.0 NON-FUNCTIONAL REQUIREMENTS	13
4.0 SYSTEM ARCHITECTURE	14
5.0 API CONTRACT & COMMUNICATION	15
Authentication Endpoints	15
User Registration	11
User Login	16
Google OAuth Login	16
User Logout	17
Create Visitor Entry (Check-in)	17
Get Active Logs	18
Process Check-out	19
Admin Void Record	19
Get Current Authenticated User	19
Get System Holiday Status	20
Get Historical Logs	20
6.0 DATABASE DESIGN	23
7.0 UI/UX DESIGN	25
8.0 PLAN	40

EXECUTIVE SUMMARY

1.1 Project Overview

Vigilo is a streamlined digital registry designed to automate the tracking of visitor arrivals and departures within micro-to-small residential properties such as mini-studio rentals, dormitories, and bed space facilities. The system replaces traditional paper-based logbooks with a controlled web-based platform and native Android companion app, enabling security personnel to efficiently record visitor details including the specific host (tenant) being visited, monitor active guests in real-time, and process check-outs with accurate timestamping

1.2 Objectives

1. Design a responsive user interface that works consistently across web and mobile platforms
2. Develop a user-friendly digital logbook that replaces paper-based visitor records.
3. Implement secure Staff authentication using email and password credentials, as well as Google OAuth integration.
4. Provide a real-time dashboard displaying all active visitors and public security alerts.
5. Ensure Staff accountability by recording which specific user authorized each visitor entry.

1.3 Scope

Included Features:

- Staff registration and authentication (Email/Password and Google OAuth)
- Visitor entry form (Name, Contact, Host Name, Room Number, Purpose, Extended Visit).
- System-generated arrival and departure timestamping.
- File upload functionality to attach digital copies of Visitor IDs to their record.
- Active visitor dashboard with Public Holidays API integration to warn guards of restricted building access days
- Automated SMTP Email sending for account verification and Admin void notifications.
- Search functionality for historical visitor records.

- Responsive web interface and native Android application.

Excluded Features:

- Hardware Barcode scanning integration.
- Webcam photo capture for visitors.
- Pre-registration portal for guests.

1.0 INTRODUCTION

1.1 Purpose

This document serves as the design specification for Vigilo. It outlines the functional requirements, core user journeys, and feature specifications to guide the development phase. This blueprint ensures that the core transactional logic (check-in and check-out) is clearly defined before implementation begins.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: Vigilo

Domain: Facility Management / Security

Primary Users:

1. Property Managers / Caretakers
2. Security Guards

Problem Statement: Paper-based visitor logbooks in residential rentals (such as mini-studios, dormitories, and bed spaces) are difficult to search during security incidents, prone to unreadable handwriting, and fail to provide caretakers or guards with real-time visibility of which non-tenants are currently inside the premises visiting specific rooms. Furthermore, physical logs lack an accountability link between the visitor and the resident hosting them, creating a security gap in unauthorized access.

Solution: A targeted digital visitor management system that replaces physical logbooks with structured data entry, host-visitor verification, and digital ID photo uploads. It enables property managers to monitor active visitors in real-time and enforces building-specific policies—such as restricted holiday access for contractors—via external API integration. The system ensures building security and

Staff accountability through role-based access, strict audit logging, and automated Admin notifications.

2.2 Core User Journeys

Journey 1: Logging a New Visitor (Check-In)

1. A Staff member logs into Vigilo using standard credentials or Google OAuth.
2. Click on "New Visitor Entry."
3. Enters visitor details (Name, Contact Number, Host Name, Visitor Type, Destination/Room, Purpose, Extended Visit) and uploads a photo of the visitor's ID.
4. Submit the form.
5. The system then saves the record with the current time and sets the visitor's status as "Active", and displays them on the Active Logs dashboard.

Journey 2: Processing a Departure (Check-Out)

1. Staff members view the "Active Logs" dashboard.
2. Locates the visitor record.
3. Click the "Check-Out" button.
4. The system records the exit timestamp, updates the visitor's status to "Checked-Out", removes the record from the Active Logs dashboard, and makes it available in the Historical Logs section for auditing.

Journey 3: Admin Voiding an Erroneous Record (RBAC & Email Integration)

1. A Staff member with the "Administrator" role logs into Vigilo.
2. Navigates from the Active Logs dashboard to the "Historical Logs" screen.
3. Locates a fraudulent or duplicated entry created by a security guard.
4. Clicks the "Void Record" action button and confirms the action.
5. The system updates the entry's status to "Voided" (performing a soft-delete to preserve the audit trail) and attaches the Admin's user ID to the record.

6. The backend automatically triggers an SMTP email to the Facility Manager summarizing the voided action for security oversight.

2.3 Feature List (MoSCoW)

MUST HAVE

1. Staff authentication (Login, Logout, Google OAuth).
2. Visitor Entry Form with File Upload (Create record).
3. Active Logs Dashboard with External API Alerts (Read records).
4. Quick Check-out function (Update record).
5. Automated SMTP Emails (Welcome emails and Void notifications).
6. Action Auditing (Tracking which Staff member created/updated records).

SHOULD HAVE

1. Basic search bar to find past visitors by name.
2. Input validation (e.g., ensuring contact numbers are digits).
3. Automated check-out (Auto-closing active logs)

COULD HAVE

1. Export logs to CSV/Excel for reporting.
2. Configurable Auto-Close Time

WON'T HAVE

1. Digital signature pad integration.
2. Real-time background checks.

2.4 Detailed Feature Specifications

Visitor Status Lifecycle

To prevent ambiguous states, visitor records strictly follow these allowed status transitions:

Current Status	Allowed Next Status	Trigger / Event
Active	Checked-Out, Auto-Closed, Voided	Visitor enters the premises.
Checked-Out	Voided	Staff manually checks out visitor.
Auto-Closed	Voided	Scheduled auto-close time is reached (default: 11:59 PM).
Voided	<i>(Terminal State - None)</i>	Admin flags record as invalid.

Feature: Staff Account Management

- **Screens:** Admin Panel -> Create Staff Account
- **Fields:** Full Name, Email Address, Role (Staff/Admin), Password
- **System Behavior:**
 - Prevents Duplicate email address
 - Creates a new Staff record in the database
 - Password is securely hashed before storage using BCrypt.
 - Sends a welcome/verification email to the newly created user via SMTP.
 - Assigns selected role for access control
 - Role determines system permissions as follows:
 - Staff Role: Can log in, create visitor entries, View Active Logs, View Historical Logs, Process check-outs
 - Admin Role: Has all Staff permissions plus ability to create Staff accounts and void visitor records

Feature: Staff Authentication

- **Screens:** Login Page.
- **Fields:** Email, Password, or "Sign in with Google" button.
- **System Behavior:**
 - Validates Staff credentials against the database or authenticates via Google OAuth.
 - The system will deny login if the Google account's email does not already exist in the users database (accounts must be pre-registered by an Admin)
 - Generates a custom JWT securely upon successful login (for both standard and OAuth methods).
 - Loads associated role and enforces role-based permissions
 - Grant access upon successful authentication
 - Denies access if credentials are invalid.

Feature: Visitor Entry Transaction

- **Screens:** Dashboard / Entry Modal.
- **Data Collected:** Full Name, Contact Number, Visitor Type (Guest/Employee/Contractor/Courier), Destination/Specific Room, Host Name (Tenant being visited), Purpose, Extended Visit boolean, and ID Attachment.
- **System Behavior:**
 - Uploads and stores the attached file (image) on the server and links the file path to the database record
 - Creates new visitor record
 - Sets its status to "Active"
 - Sets time_in to Current Timestamp
 - Sets created_at to Current Timestamp
 - Records the visitor record created_by as Logged-in user ID.

Feature: Active Logs Dashboard

- **Screens:** Main Dashboard.
- **Display:** A data table showing only visitors whose status is 'Active'. Includes columns for Name, Time In, and an action button for Check-out.
- **External API Integration:** Consumes a public API (e.g., Public Holidays/Events API) to display a banner at the top of the dashboard warning guards if today is a restricted access day.

Feature: Check-Out Processing

- **Screens:** Main Dashboard.
- **System Behavior:**
 - Updates the specific visitor's database row
 - Sets status to "Checked-Out"
 - Sets time_out to Current Timestamp
 - Sets updated_at to Current Timestamp
 - Records updated_by as Logged-in user ID

Feature: Admin Log Management (Voiding)

- **Screens:** Admin Dashboard / Log History.
- **Functions:** Allows users with an Admin role to select a fraudulent or erroneous entry and can void records regardless of current status, soft-deleting the record to preserve the audit trail.
- **Access Control:** Restricted to Administrator role only.
 - When voiding a record, the System:
 - Sets updated_at to Current Timestamp
 - Records updated_by as Logged-in Admin user ID
 - Sends an automated system notification email via SMTP to the facility manager summarizing the voided action.

- Leaves the time_out field unchanged (e.g., remains NULL if the record was Active) to accurately preserve the original transactional state.

Feature: Scheduled Auto-Close Process

- **Process:** Executes daily at a system-defined cut-off time (default: 11:59 PM).
- **System Behavior:**
 - Identifies all logs where time_out IS NULL at 11:59 PM.
 - If is_extended_visit is false, the system automatically sets the time_out to 11:59 PM.
 - The system does not restrict late-night check-ins. Any non-extended visit that remains active at the scheduled cut-off time will be auto-closed, regardless of how recently it was created.
 - Sets the visitor status to "Auto-Closed".
 - Sets auto_closed = true and updated_by = "SYSTEM" to maintain data integrity and distinguish automated cleanups from manual staff check-outs.
- **Logic Rule:** The is_extended_visit boolean acts as an "Exemption Flag" for the 11:59 PM cleanup.
- **Standard Behavior:** For normal visitors, is_extended_visit is false; they are automatically transitioned to status = "Auto-Closed" at 11:59 PM.
- **Extended Behavior:** If is_extended_visit is true, the record remains Active past midnight and must be manually checked out by staff.

2.5 Acceptance Criteria

AC-1: Successful Visitor Check-In

- Given I am a logged-in Staff member
- When I submit the new visitor form with valid details and a valid ID image file
- Then the system stores the following:
 - time_in timestamp
 - created_by user ID
 - host_name
 - visitor_type_id
 - location_id
 - status = "Active"

- file_url path
- "Extended Visit = true (or false)"
- And the visitor should appear on the Active Logs dashboard

AC-2: Successful Visitor Check-Out

- Given I am viewing the Active Logs dashboard
- When I click the "Check-Out" on a visitor record
- Then that visitor should be removed from the active list
- And the system must record the following:
 - time_out timestamp
 - updated_by user ID
 - status = "Checked-Out"

AC-3: Admin Log Voiding

- Given I am logged in as an Administrator
- When I flag a visitor entry as erroneous or fraudulent
- Then the record's status should change to "Voided"
- And the record remains stored for audit purposes
- And the system sends a notification email via SMTP to the facility manager.

AC-4: Scheduled Auto-Close Process

- Given there are visitor records with an "Active" status
- When the system executes the scheduled auto-close process at the defined cut-off time
- Then records not marked as "Extended Visit" are updated as follows:
 - status = "Auto-Closed"
 - time_out = 11:59 PM (system cut-off time)
 - Updated_by = SYSTEM
 - auto_closed = true
- And they no longer appear in Active Logs.

AC-5: Google OAuth Login

- Given I am a Staff member with a registered Google account
- When I click "Sign in with Google" and authenticate
- Then the system must verify my identity with Google
- And generate a custom JWT for my session
- And grant me access to the dashboard.

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API response time: ≤ 2 seconds for 95% of requests under normal load
- Web page load time: ≤ 3 seconds on standard broadband connection
- System supports up to 100 concurrent authenticated users
- Database queries for visitor search complete within 500ms
- Scheduled Auto-Close process completes within 60 seconds

3.2 Security Requirements

- HTTPS for all communications
- Staff authentication implemented using secure JWT-based session management
- Password hashing with BCrypt (salt rounds = 12)
- Protection against SQL injection
- Uploaded files must be validated to accept only safe image/document formats with a maximum file size of 5MB.
- XSS protection via input sanitization and output encoding
- Rate limiting enforced at 100 requests per minute per IP
- Audit fields (created_by, updated_by, created_at, updated_at) must not be manually editable

3.3 Compatibility Requirements

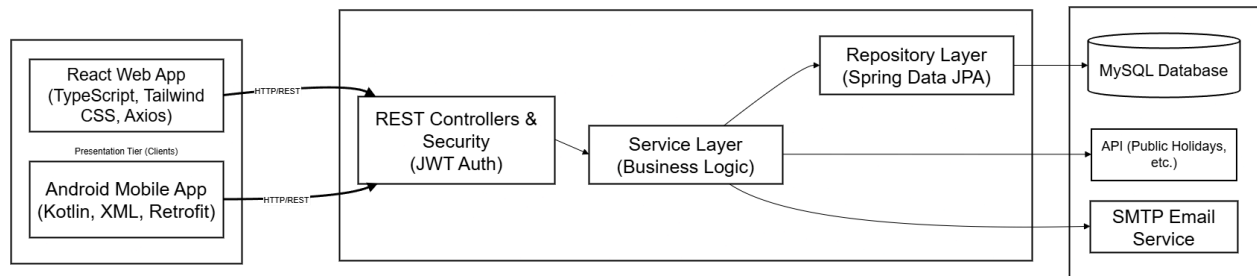
- **Web Browsers:** Chrome, Firefox, Safari, Edge (latest 2 versions)
- **Android:** API Level 34 (Android 14) required for the mobile companion app using XML-Based UI Layouts.
- **Screen Sizes:** Mobile (360px+), Tablet (768px+), Desktop (1024px+)
- **Operating Systems:** Windows 10+, macOS 10.15+, Linux Ubuntu 20.04+

3.4 Usability Requirements

- A new Staff user should be able to complete a visitor check-in within 3 minutes without training.
- Interfaces must maintain a consistent navigation layout across all screens.
- Error messages must clearly indicate the issue and provide recovery instructions if necessary.
- Form validation errors must be displayed in real time before submission
- Touch targets must be minimum 44x44px on mobile devices
- System must support full keyboard navigation

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram



Technology Stack:

- **Backend:** Java 17, Spring Boot 3.x, Spring Security, Spring Data JPA
- **Database:** [PostgreSQL 14+](#)
- **Web Frontend:** React 18, TypeScript, Tailwind CSS, Axios
- **Mobile:** Kotlin, Retrofit
- **Build Tools:** Maven (Backend), npm/yarn (Web), Gradle (Android)
- **Deployment:** [Supabase \(Database\)](#), Render (Web), APK (Mobile)

Architecture Pattern:

The backend follows a Layered Architecture consisting of a REST Controller layer, Service layer (Business Logic), Repository layer (Spring Data JPA), DTO mapping layer, Security configuration, and Global Exception Handling. This ensures a strict separation of concerns, scalability, and code maintainability.

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Base URL	https://[server_hostname]:[port]/api/v1
Format	JSON for all requests/responses
Authentication	Bearer token (JWT) in Authorization header
Response Structure	<pre>{ "success": <boolean>, "data": <object null>, "error": { "code": <string>, "message": <string>, "details": <object null> }, "timestamp": <string> }</pre>

5.2 Endpoint Specifications

Authentication Endpoints

User Registration

Description	Creates a new Staff or Admin account.
API URL	/auth/register
HTTP Request Method	POST
Format	JSON
Authentication	Bearer token (Admin JWT)
Request Payload	<pre>{ "email": "<email>", "password": "<password>", "firstName": "<firstName>", "lastName": "<lastName>", "role": "<role>" }</pre>
Response Structure	<pre>{ "success": true, "data": { "user": { "email": "<email>", "firstName": "<firstName>", "lastName": "<lastName>", "role": "<role>" } } }</pre>

	<pre> }, "error": null, "timestamp": "<timestamp>" } </pre>
--	---

User Login

Description	Authenticates a user and returns a JWT for session management.
API URL	/auth/login
HTTP Method	POST
Format	JSON
Authentication	None
Request Payload	<pre> { "email": "<email>", "password": "<password>" } </pre>
Response Structure	<pre> { "success": true, "data": { "user": { "id": "<number>", "email": "<email>", "firstName": "<firstName>", "lastName": "<lastName>", "role": "<role>" }, "accessToken": "<token>" }, "error": null, "timestamp": "<timestamp>" } </pre>

Google OAuth Login

Description	Authenticates a Staff member using their Google account and returns a custom JWT.
API URL	/auth/google
HTTP Method	POST
Format	JSON
Authentication	None
Request Payload	<pre> { "idToken": "<google_id_token>" } </pre>
Response Structure	<pre> { </pre>

	<pre> "success": true, "data": { "user": { "email": "<email>", "firstName": "<firstName>", "lastName": "<lastName>", "role": "<role>" }, "accessToken": "<token>" }, "error": null, "timestamp": "<timestamp>" } </pre>
--	---

User Logout

Description	Ends the user session client-side by removing the JWT from storage. No server API request is needed.
API URL	N/A (Client-side only)
HTTP Method	N/A
Format	N/A
Authentication	N/A
Request Payload	None
Response Structure	None

Create Visitor Entry (Check-In)

Description	Creates a new active visitor log and uploads their ID photo to the server.
API URL	/logs/check-in
HTTP Method	POST
Format	multipart/form-data
Authentication	Bearer token (JWT)
Request Payload	<pre> { "fullName": "<string>", "contactNumber": "<string>", "hostName": "<string>", "visitorTypeId": <int>, "locationId": <int>, "purpose": "<string>", "isExtendedVisit": <bool>, "idImages": [<binary>] } </pre>

	}
Response Structure	<pre>{ "success": true, "data": { "logId": "<logId>", "status": "<status>", "attachments": [{ "fileUrl": "<fileUrl>", "fileType": "<fileType>" }], "timeIn": "<timestamp>" }, "error": null, "timestamp": "<timestamp>" }</pre>

Get Active Logs

Description	Retrieves a list of all visitors currently marked as "Active" for the real-time dashboard.
API URL	/logs/active
HTTP Method	GET
Format	JSON
Authentication	Bearer token (JWT)
Request Payload	None
Response Structure	<pre>{ "success": true, "data": [{ "logId": "<logId>", "fullName": "<fullName>", "hostName": "<hostName>", "locationId": "<locationId>", "timeIn": "<timestamp>" }], "error": null, "timestamp": "<timestamp>" }</pre>

Process Check-Out

Description	Updates an active visitor's status to "Checked-Out" and records the exit timestamp.
API URL	/logs/{id}/check-out
HTTP Method	PUT
Format	JSON
Authentication	Bearer token (JWT)
Request Payload	None
Response Structure	<pre>{ "success": true, "data": { "logId": "<logId>", "status": "<status>", "timeOut": "<timestamp>" }, "error": null, "timestamp": "<timestamp>" }</pre>

Admin Void Record

Description	Flags an erroneous record as "Voided" and triggers an SMTP notification email.
API URL	/logs/{id}/void
HTTP Method	PUT
Format	JSON
Authentication	Bearer token (Admin JWT Only)
Request Payload	None
Response Structure	<pre>{ "success": true, "data": { "logId": "<logId>", "status": "<status>", "timeOut": "<timestamp>" }, "error": null, "timestamp": "<timestamp>" }</pre>

Get Current Authenticated User

Description	Retrieves the profile information of the currently authenticated Staff member using the JWT session.
API URL	/auth/me

HTTP Method	GET
Format	JSON
Authentication	Bearer token (JWT)
Request Payload	None
Response Structure	<pre>{ "success": true, "data": { "user": { "id": "<number>", "email": "<email>", "firstName": "<firstName>", "lastName": "<lastName>", "role": "<role>" } }, "error": null, "timestamp": "<timestamp>" }</pre>

Get System Holiday Status

Description	Retrieves the current holiday status to enforce building access policies.
API URL	/system/holiday-status
HTTP Method	GET
Format	JSON
Authentication	Bearer token (JWT)
Request Payload	None
Response Structure	<pre>{ "success": true, "data": { "isHoliday": true, "holidayName": <holiday> }, "error": null, "timestamp": "<timestamp>" }</pre>

Get Historical Logs

Description	Retrieves paginated historical visitor records (Checked-Out, Auto-Closed, Voided) for auditing.
API URL	/logs/history
HTTP Method	GET
Format	JSON
Authentication	Bearer token (JWT)

Request Payload	None
Response Structure	<pre> { "success": true, "data": { "content": [{ "logId": "<logId>", "fullName": "<fullName>", "status": "<status>", "timeIn": "<timestamp>", "timeOut": "<timestamp>" }], "totalPages": "<number>" }, "error": null, "timestamp": "<timestamp>" } </pre>

5.3 Error Handling

HTTP Status Codes

- **200 OK** - Successful request
- **201 Created** - Resource created
- **400 Bad Request** - Invalid input
- **401 Unauthorized** - Authentication required/failed
- **403 Forbidden** - Insufficient permissions
- **404 Not Found** - Resource doesn't exist
- **409 Conflict** - Duplicate resource
- **500 Internal Server Error** - Server error

Error Code Examples

Example 1: Authentication Failure	<pre> { "success": false, "data": null, "error": { </pre>
--------------------------------------	---

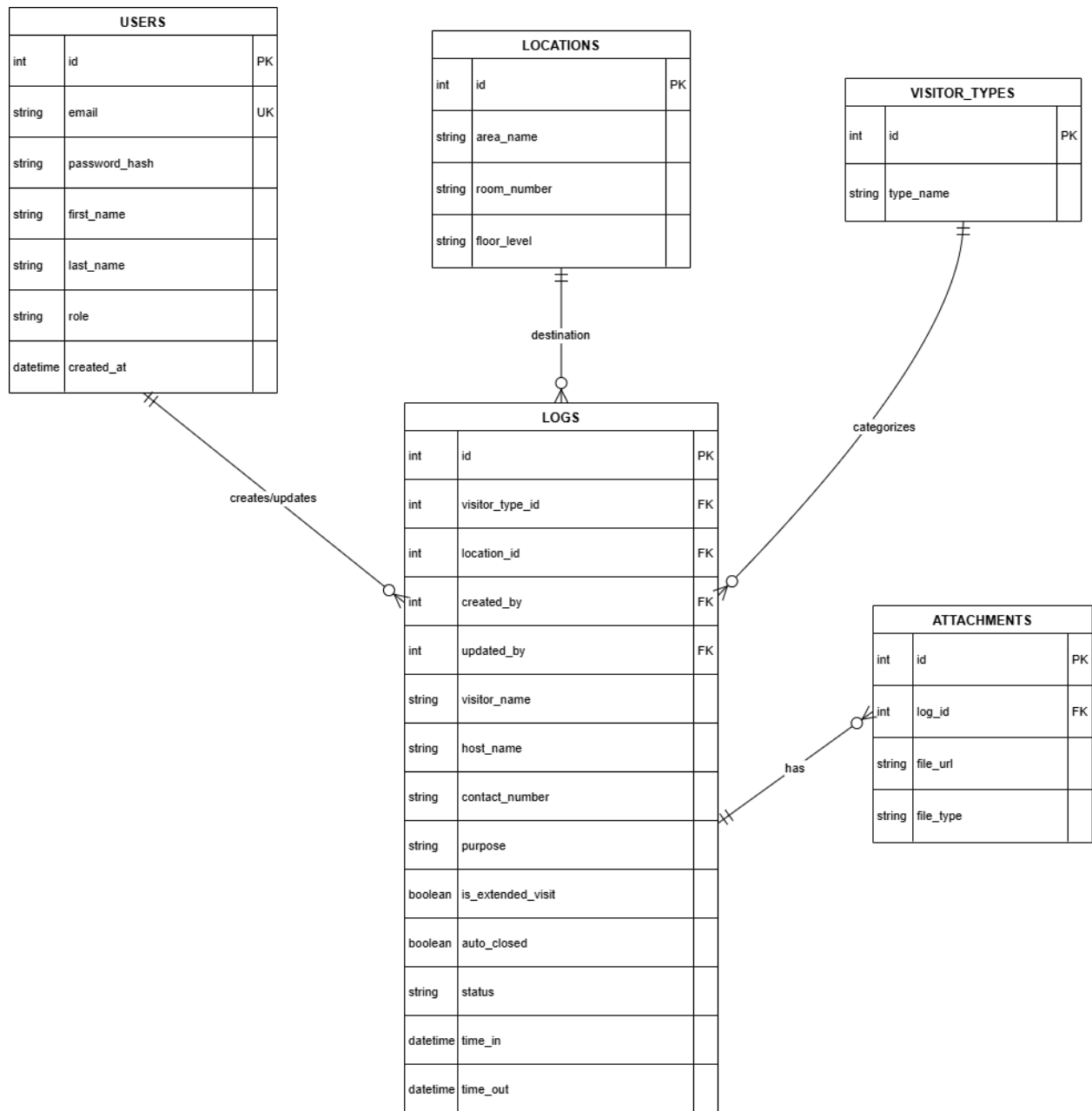
	<pre> "code": "AUTH-001", "message": "Invalid credentials", "details": "Email or password is incorrect" }, "timestamp": "2026-03-01T10:30:00Z" } </pre>
Example 2: Form Validation Failure	<pre> { "success": false, "data": null, "error": { "code": "VALID-001", "message": "Validation failed", "details": { "fullName": "Full name is required", "contactNumber": "Must be a valid 11-digit number" } }, "timestamp": "2026-03-01T10:30:00Z" } </pre>

Common Error Codes

- **AUTH-001:** Invalid credentials
- **AUTH-002:** Token expired
- **AUTH-003:** Insufficient permissions
- **VALID-001:** Validation failed
- **DB-001:** Resource not found
- **DB-002:** Duplicate entry
- **VIGILO-001:** Visitor is already checked in and active
- **SYSTEM-001:** Internal server error

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram



Detailed Relationships:

- **One-to-Many: Users → Logs**: Each Staff member (Admin/Staff) is linked to the visitor records they create, ensuring full system accountability.
- **One-to-Many: Locations → Logs**: Every visit is tied to a specific physical destination, allowing for granular tracking by room or office number .

- **One-to-Many: Visitor_Types → Logs:** Provides a standardized lookup for categories such as Guest, Employee, or Contractor .
- **One-to-Many: Logs → Attachments:** A single visit record can store multiple files, specifically the mandatory visitor ID images

Key Tables:

1. **users:** Stores Staff credentials, BCrypt hashed passwords, and assigned RBAC roles .
2. **locations:** A lookup table for specific facility areas, including room names, numbers, and floor levels.
3. **visitor_types:** Standardizes categories to prevent data duplication and ensure consistent reporting.
4. **logs:** The core business module storing visitor identity, automated timestamps, and visit status .
5. **attachments:** Maintains metadata and server paths for verified visitor ID file uploads .

Table Structure Summary:

- **users:** id, email, password_hash, first_name, last_name, role, created_at.
- **locations:** id, area_name, room_number, floor_level
- **visitor_types:** id, type_name (e.g., Contractor, Courier).
- **logs:** id, visitor_name, host_name, visitor_type_id (FK), location_id (FK), purpose, is_extended_visit (boolean), auto_closed (boolean), status, time_in, time_out, created_by (FK), updated_by (FK).
- **attachments:** id, log_id (FK), file_url, file_type, uploaded_at .

7.0 UI/UX DESIGN

7.1 Web Application Wireframes

Active Logs Dashboard Page (Staff View)

Main Dashboard

logo

Vigilo

Visitor Management

Active Logs Button

Historical Logs Button

Public API banner warning message

Search bar

Staff Badge John Doe
Title

New Visitor Check-in

Active Logs

Currently monitoring n active visitors on the premises.

Visitor Name	Type	Host (Tenant)	Destination/Room	Time in	Action	
John Doe	Contractor	John Doe	Basement	Today, 08:30 AM	Check out	
John Doe	Guest	John Doe	Room 20	Today, 10:30 AM	Check out	
John Doe	Extended	Guest	John Doe	Room 21	Yesterday, 7:45 PM	Check out

Showing 3 of 3 active visitors

Active Logs Dashboard Page (Admin View)

Main Dashboard

logo

Vigilo

Visitor Management

Active Logs Button

Historical Logs Button

System

Admin Dashboard Button

Public API banner warning message

Search bar

Admin Badge John Doe
Title

New Visitor Check-in

Active Logs

Currently monitoring n active visitors on the premises.

Visitor Name	Type	Host (Tenant)	Destination/Room	Time in	Action	
John Doe	Contractor	John Doe	Basement	Today, 08:30 AM	Check out	
John Doe	Guest	John Doe	Room 20	Today, 10:30 AM	Check out	
John Doe	Extended	Guest	John Doe	Room 21	Yesterday, 7:45 PM	Check out

Showing 3 of 3 active visitors

Historical Logs Dashboard Page (Staff View)

Historical Logs Dashboard

logo

Vigilo

Visitor Management

Active Logs Button

Historical Logs Button

Public API banner warning message

Search bar

Staff BadgeJohn DoeTitle

Historical Logs

Audit trail of all past visitor arrivals and departures

All Statuses

Last 7 Days

Visitor Name	Host and Destination	Time in	Time out	Status
John Doe Guest	John Doe Room 32	Feb. 28, 02:30 AM	Today, 08:30 AM	Checked out
John Doe Courier	John Doe Room 31	Feb. 27, 08:30 AM	Today, 10:30 AM	Auto-Closed by SYSTEM
John Doe Contractor	Facility Manager Roof Access	Feb. 26, 7:45 PM	N/A	Voided

Showing 3 of 1,234 historical records

Historical Logs Dashboard Page (Admin View)

Historical Logs Dashboard

logo

Vigilo

Visitor Management

Active Logs Button

Historical Logs Button

System

Admin Dashboard Button

Public API banner warning message

Search bar

Admin BadgeJohn DoeTitle

Historical Logs

Audit trail of all past visitor arrivals and departures

All Statuses

Last 7 Days

Visitor Name	Host and Destination	Time in	Time out	Status	Action
John Doe Guest	John Doe Room 32	Feb. 28, 02:30 AM	Today, 08:30 AM	Checked out	Void Record
John Doe Courier	John Doe Room 31	Feb. 27, 08:30 AM	Today, 10:30 AM	Auto-Closed by SYSTEM	Void Record
John Doe Contractor	Facility Manager Roof Access	Feb. 26, 7:45 PM	N/A	Voided	No Action

Showing 3 of 1,234 historical records

Staff Management Dashboard Page (Admin View)

Staff Management Dashboard

logo

Vigilo

Visitor Management

Active Logs Button

Historical Logs Button

System

Admin Dashboard Button

Public API banner warning message

Search bar

Admin Badge John Doe Title

Staff Management

Location Management

System Audit

Staff Accounts

Add or Manage guard accounts for the facility

Register New Staff

Staff Name	Email Address	Role	Status
John Doe	JohnDoe@facility.com	Admin	Offline
John Doe	JohnDoe2@facility.com	Admin	Active

Facility Management Dashboard Page (Admin View)

Staff Management Dashboard

logo

Vigilo

Visitor Management

Active Logs Button

Historical Logs Button

System

Admin Dashboard Button

Public API banner warning message

Search bar

Admin Badge John Doe Title

Staff Management

Location Management

System Audit

Facility Locations

Configure the official list of rooms and areas for visitor check-in.

Add New Location

Admin Note: These locations are used to populate the Searchable dropdown on the Staff's check-in form.

Area/Section	Room Number	Floor Level	Actions
Roof Access	RA-1	4th Floor	Offline
Dormitory	12	2nd Floor	Active

System Audit Dashboard Page (Admin View)

System Audit Dashboard

logo

Vigilo

Visitor Management

Active Logs Button

Historical Logs Button

System

Admin Dashboard Button

Public API banner warning message

Search bar

Admin BadgeJohn DoeTitle

Staff ManagementLocation ManagementSystem Audit

System Audit Log

A chronological record of all critical system actions

Admin Note: These locations are used to populate the Searchable dropdown on the Staff's check-in form.

Timestamp	User	Action	Details
Mar. 1, 2025, 11:59:00 PM	SYSTEM	Auto-Close Executed	Automatically closed 5 active logs at system cut-off time.
Mar. 1, 2025, 04:32:15 PM	John Doe (Admin)	Record Voided	Voided visitor record #1234 (Duplicate Entry).
Feb 10, 2025, 10:00:05 AM	John Doe (Admin)	User Registered	Created new Staff account for 'John Guard'(j.guard@facility.com)
Feb 1, 2025, 08:15:30 AM	John Doe (Guard)	Visitor Check-in	Logged new active visitor: 'Jane Doe' visiting Room 14.

Login Page

Login

logo

Vigilo

Sign in to your account to continue

Invalid email or password. Please try again.

Email Address

staff@facility.com

Email AddressForgot Password?

Login

or continue with

Sign in with Google

Logout Modal

Logout modal

icon

Ready to Leave?

Are you sure you want to log out of your session?
You will need to sign in again to access the dashboard.

Cancel

Logout

Visitor Check-in modal

Visitor Check-in modal

New Visitor Entry

First Name

John

Last Name

Doe

Host Name (Tenant)

Jane

Destination/Room

Search Bar Dropdown

Visitor Type

Dropdown

Mark as Extended Visit

Switch

Exempts this visitor from the automated 11:59 PM system checkout.

Purpose of Visit

Dropdown

Visitor ID Attachment

File Upload

Cancel

Submit Check-in

Create Staff Account Modal (Admin View)

Admin create staff account modal

Register New Staff X

First Name

John

Last Name

Doe

Email Address

JohnDoe@facility.staff

Temporary Password

Cancel

Register

Add Facility Location Modal (Admin View)

Admin create staff account modal

Add New Facility Location X

Area Name / Building Section

e.g. Basement

Room Number

e.g. 1

Floor Level

e.g. 2nd Floor

Note: Adding this location makes it instantly searchable for staff during visitor check-in.

Cancel

Save Location

Login Screen

- **Layout:** A clean, centered authentication card over a subtle background.
- **Header:** Vigilo Logo and a "Sign in to your account" heading.
- **Input Fields:** Email Address and Password.
- **Primary Action:** A solid "Login" button.
- **Secondary Action:** A prominent "Sign in with Google" button with the Google logo for OAuth authentication.
- **Feedback:** Inline error messages for invalid credentials (e.g., "Invalid email or password").

Staff Management & Registration (Admin Role Only)

- **Layout:** Utilizes the base dashboard layout (Sidebar + Header) with the "Admin Settings" tab active.
- **Staff Data Table:** A grid listing all registered system users (Name, Email, Role, Date Created).
- **Create Staff Form (Modal):** The official "Registration" form for the system.
 - *Fields:* Full Name, Email Address, Role (Dropdown: Admin/Staff), and Password.
 - *Actions:* "Cancel" and "Register User" buttons.

Logout Flow

- **Trigger:** Accessed via a dropdown menu when clicking the User Profile icon in the top right Header.
- **Confirmation Modal:** A small pop-up asking, "Are you sure you want to log out of your session?"
- **Actions:** "Cancel" (returns to dashboard) and "Logout" (clears JWT and redirects to Login Screen).

Active Logs Dashboard (Main Screen)

- **Header:** Vigilo Logo, Global Search Bar (find visitors by name), Public Holidays Alert Banner (consuming the external API), and User Profile/Logout menu.
- **Sidebar Navigation:** Active Logs, Historical Logs, Admin Settings (if Admin role).
- **Primary Action:** A prominent "New Visitor Check-In" button.
- **Active Table:** A data grid displaying visitors with an "Active" status.

- *Columns:* Visitor Name, Visitor Type, Host Name (Tenant), Destination/Room, and Time In.
- *Row Action:* A dedicated "Check-Out" button for each active visitor.

Visitor Check-In Form (Modal/Overlay)

- **Form Title:** New Visitor Entry
- **Input Fields:**
 - Full Name & Contact Number (Text Inputs)
 - Host Name / Tenant (Text Input)
 - Visitor Type (Dropdown: Guest, Contractor, Courier, etc.)
 - Destination/Room (Searchable Dropdown)
 - Purpose of Visit (Text Area)
- **Toggle:** "Extended Visit" (Switch/Checkbox) to exempt the visitor from the 11:59 PM auto-close.
- **File Attachment:** Drag-and-drop area or "Upload File" button for the Visitor ID photo.
- **Actions:** "Cancel" and "Submit Check-In" buttons.

Historical Logs Screen

- **Layout:** Similar to the Active Dashboard, but displaying visitors with "Checked-Out", "Auto-Closed", or "Voided" statuses.
- **Data Table Columns:** Includes 'Time Out' and 'Status' indicators.
- **Admin Features:** If logged in as Admin, a "Void Record" action button appears next to standard entries.

Staff Management (Admin Only)

- **Purpose:** Provides a centralized interface for the Administrator to manage security personnel accounts and assign system roles.

- **Table Columns:** Full Name, Email Address, Role (Staff/Admin), and Date Created.
- **Primary Action:** "Register New Staff" button which triggers the registration modal.
- **Registration Form (Modal):** * **Fields:** Full Name, Email Address, Role (Dropdown), and Password.
 - **Actions:** "Cancel" and "Register User" buttons.

Location Management (Admin Only)

- **Purpose:** Provides a centralized interface for the Administrator to maintain the master list of valid rooms and areas.
- **Table Columns:** Area Name, Room Number, and Floor Level.
- **Primary Action:** "Add New Location" button which triggers a modal for pre-configuring facility destinations.
- **System Logic:** These entries are used to populate the **Searchable Autocomplete** field in the Staff Check-In form, preventing duplicate or erroneous data entry.

System Audit Log (Admin Only)

- **Purpose:** Fulfills the "Action Auditing" requirement by providing a non-editable, chronological timeline of critical system events.
- **Data Table Columns:** Timestamp, User (Staff/Admin/SYSTEM), Action Performed, and Details.
- **Logged Events:** * Staff account creation and role assignments.
 - Admin "Void" actions on visitor records.
 - Automated "System" events, such as the 11:59 PM scheduled auto-close.

7.2 Mobile Application Wireframes

Login Page

iPhone 13 & 14 - 3



Vigilo

Sign in to your account to continue

Invalid email or password. Please try again.

Email

staff@facility.com

Email

Forgot Password?

Login

or continue with

Sign in with Google

Logout Modal

Logout modal



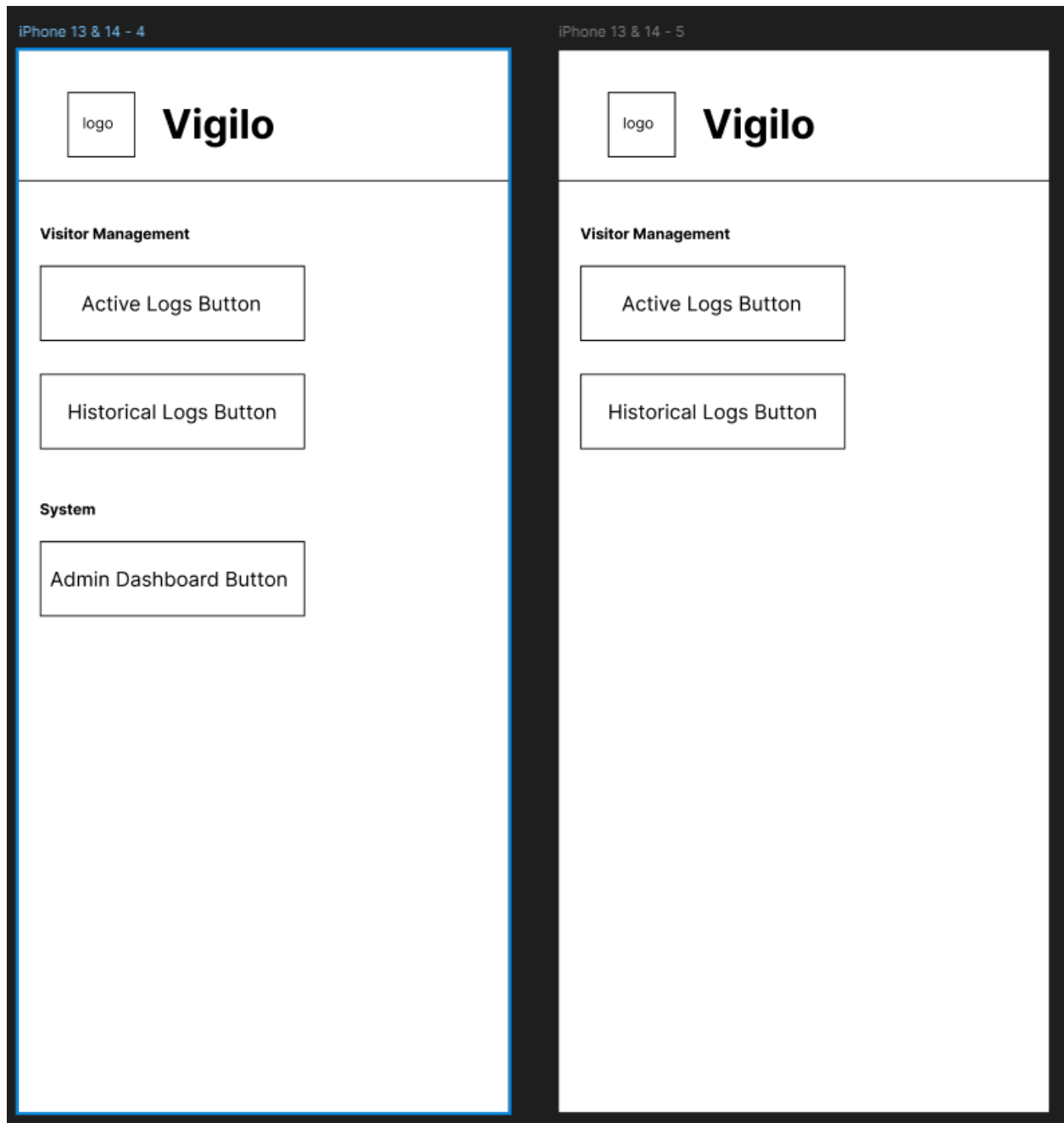
Ready to Leave?

Are you sure you want to log out of your session?
You will need to sign in again to access the dashboard.

Cancel

Logout

Sidebar (Staff and Admin)



Visitor Check-in modal

Visitor Check-in modal

New Visitor Entry

First Name

John

Last Name

Doe

Host Name (Tenant)

Jane

Destination/Room

Search Bar Dropdown

Visitor Type

Dropdown

Purpose of Visit

Dropdown

Visitor ID Attachment

File Upload

Mark as Extended Visit

Switch

Exempts this visitor from the automated 11:59 PM system checkout.

Cancel

Submit Check-in

Create Staff Account Modal (Admin View)

Admin create staff account modal

Register New Staff X

First Name

Last Name

Email Address

Temporary Password

Add Facility Location Modal (Admin View)

Admin create staff account modal

Add New Facility Location X

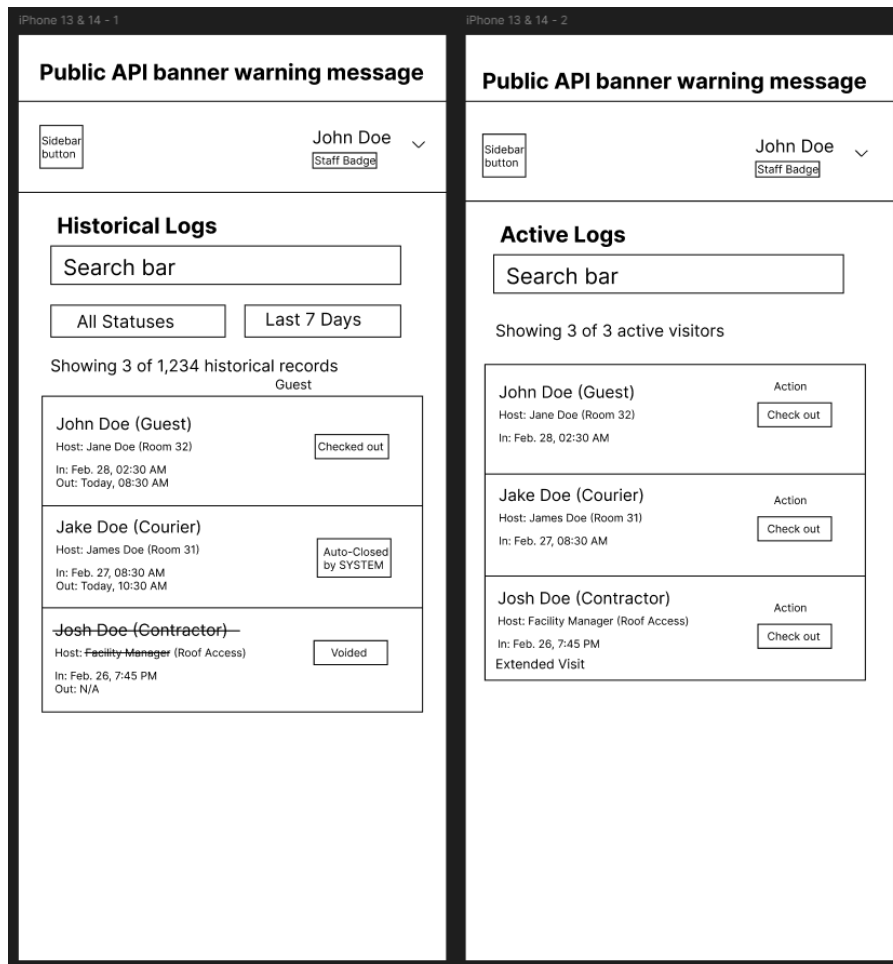
Area Name / Building Section

Room Number

Floor Level

Note: Adding this location makes it instantly searchable for staff during visitor check-in.

Active Logs Dashboard and Historical Logs Dashboard



Mobile Home Screen (Active Logs)

- **Layout:** Optimized for one-handed use by a security guard.
- **Header:** Quick Search bar to find active visitors.
- **Active List:** A vertically scrolling list (RecyclerView) of active guests.
 - *Gesture Support:* Swipe right on a visitor card to quickly reveal the "Check-Out" action.
- **Floating Action Button (FAB):** A large "+" button in the bottom right corner for immediate Check-In entry.
- **Bottom Navigation:** Icons for Active Logs, History, and Profile/Settings.

Mobile Check-In Form Screen

- **Input Optimization:** Simplified fields with large, touch-friendly inputs (minimum 44x44px target sizes).

- **Camera Integration:** A direct "Take Photo of ID" button that triggers the native Android camera or file picker.
- **Sticky Action:** A fixed "Submit Check-In" button at the bottom of the screen.

Mobile Login Screen

- **Layout:** Centered, distraction-free authentication view.
- **Header:** Vigilo Logo and "Guard Sign-In".
- **Inputs:** Email and Password fields optimized for mobile keyboards.
- **Actions:** A large, primary "Login" button and a secondary "Sign in with Google" button.

Mobile Historical Logs Screen

- **Access:** Reached via the "History" icon in the Bottom Navigation.
- **Header:** Search bar specifically for querying past visitors by name.
- **List View:** A scrolling list (RecyclerView) of past visitors displaying their final status (Checked-Out, Auto-Closed, Voided) via color-coded badges.

Mobile Profile & Settings (Logout Flow)

- **Access:** Reached via the "Profile" icon in the Bottom Navigation.
- **Content:** Displays the current authenticated Staff member's Name, Email, and Role.
- **Primary Action:** A prominent, full-width red "Logout" button that clears the local JWT and returns the user to the Login Screen.

Design System:

- **Colors:** Primary Facility Blue (#2563EB), Secondary (#4B5563), Success Green (#10B981) for Active status, Error Red (#EF4444) for Voided/Alerts.
- **Typography:** Inter font family, highly legible for security staff.
- **Spacing:** 8px grid system for consistent component padding.
- **Responsive:** Mobile-first approach for the web dashboard, with breakpoints at 640px, 768px, and 1024px.

8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1-2)

Week 1: Requirements & Architecture

- Day 1-2: Project setup, finalize FRS and NFR.
- Day 3-5: System architecture design and External API selection.

Week 2: Detailed Design

- Day 1-2: API specification and JSON contracts.
- Day 3-4: 5-table Database Design (ERD creation).
- Day 5-7: UI/UX Figma wireframes for Web and Mobile.
-

Phase 2: Backend Development (Week 3-4)

Week 3: Security & Core Setup

- Day 1-2: Spring Boot setup, MySQL database configuration.
- Day 3-5: Spring Security implementation, JWT generation, and Google OAuth integration.

Week 4: Vigilo Core Features

- Day 1-2: Visitor CRUD operations and Check-in/Check-out logic.
- Day 3: File upload service.
- Day 4: External Public API integration & Scheduled Cron Job
- Day 5: SMTP Email service setup

Phase 3: Web Application (Week 5-6)

Week 5: React Foundation

- Day 1-2: React setup, Tailwind CSS, Axios.
- Day 3-5: Login screen, Google OAuth flow, and Base Dashboard layout.

Week 6: Web Feature Completion

- Day 1-2: Active Logs data table, Check-in modal with file upload handling.
- Day 3: Historical logs and Admin voiding interface.
- Day 4-5: Responsive design polish and API testing.

Phase 4: Mobile Application (Week 7-8)

Week 7: Android Foundation

- Day 1-2: Android Studio setup, XML Layouts , Retrofit configuration.
- Day 3-5: Authentication screens, secure JWT storage.

Week 8: Mobile Core Integration

- Day 1-2: Active logs RecyclerView, Check-out API consumption.
- Day 3-4: Camera intent / File picker for ID upload, Check-in form API consumption.
- Day 5: Role awareness UI adjustments and emulator testing.

Phase 5: Integration & Deployment (Week 9-10)

Week 9: Integration Testing

- Day 1-3: End-to-end testing (Web -> Backend, Mobile -> Backend).
- Day 4-5: Security review and bug fixes.

Week 10: Deployment & Defense

- Day 1: Web app deployment (Render).
- Day 2-4: Final Testing and Bug Fixes.
- Day 5: Project Submission.

Milestones:

- **M1 (End Week 2):** All design documents (ERD, Wireframes, SDD) complete.
- **M2 (End Week 4):** Backend API fully functional with SMTP, Uploads, and OAuth.
- **M3 (End Week 6):** React Web application complete and connected to API.

- **M4 (End Week 8):** Android Mobile application complete.
- **M5 (End Week 10):** Full system integrated, and staged locally.

Critical Path:

1. Authentication & JWT System (Week 3)
2. Visitor Logging & File Upload API (Week 4)
3. React Web Dashboard (Week 5-6)
4. Android Retrofit Integration (Week 7-8)

Risk Mitigation:

- Start with the simplest working version of each feature before adding complexity.
- Test API integration points via Postman before connecting the frontend clients.
- Keep regular Git backups of working versions to prevent data loss.
- Prioritize core Check-in/Check-out functionality over NFRs if time becomes constrained.